MODULE ONE

- **Introduction to Operating Systems**
  - Operating System (OS)
    - An operating system is system software that manages computer hardware and provides services for programs
      - Acts as an intermediary between users/applications and hardware
  - Main goals
    - Manage hardware resources such as CPU, memory, storage, and devices
      - Provide an interface for programs and users
      - Ensure efficient and secure execution of programs
  - Examples
    - Linux
    - Windows
    - macOS
- **Operating System Services**
  - Major OS Services overview
    - Operating systems provide services to make program execution convenient and efficient
      - Services cover execution, I/O, file management, communication, error handling, allocation, accounting, and security
  - Program Execution and I/O
    - Program execution service loads a program into memory and runs it
      - I/O operations provide a standard way for programs to interact with devices like disks, keyboards, and printers
  - File, Communication, and Error Services
    - File system manipulation lets programs create, read, write, and delete files
    - Communication services enable interprocess information exchange
    - Error detection identifies and handles hardware and software errors
  - Resource and Protection Services
    - Resource allocation distributes CPU time, memory, and other resources among processes
    - Accounting tracks resource usage
    - Protection and security control access to system resources and ensure system safety
- **Overview of Operating Systems and Kernels**
  - Operating System
    - The operating system is the complete software that manages computer resources and provides services to users
  - Kernel
    - The kernel is the core component of an operating system that directly interacts with hardware
      - Runs in privileged mode to control hardware safely
  - Kernel responsibilities
    - Process management
    - Memory management
    - Device management
    - System calls
    - Interrupt handling
- **Linux Versus Classic Unix Kernels**
  - Unix Kernels
    - Original operating systems developed in the 1970s with monolithic design
      - Commercial implementations include Solaris, AIX, and HP-UX
  - Linux Kernel
    - Open-source Unix-like kernel created by Linus Torvalds in 1991
      - Monolithic but modular; widely used in servers, embedded systems, and Android
  - Key differences
    - Source code: proprietary for classic Unix versus open s urce for Linux
    - Development: controlled by companies for classic Unix versus community driven for Linux
    - Modularity: limited in classic Unix versus highly modular in Linux
    - Hardware support: limited in classic Unix versus very broad in Linux
- **Process Concepts**
  - Definition of a process
    - A process is a program that is currently executing
  - Components of a process
    - Program code and data
    - Program counter, stack, heap, and CPU registers
  - Concurrency
    - Processes allow multiple programs to run concurrently on a system
- **Process Creation**
  - How processes are created
    - A new process is created when a running process requests the OS to start another process
  - Unix/Linux mechanisms
    - fork creates a new child process while the parent continues execution
    - exec replaces the current process memory with a new program
- **Process States**
  - Common process states
    - New: process is being created
    - Ready: waiting for CPU time
    - Running: currently executing on CPU
    - Waiting (Blocked): waiting for I/O or an event
    - Terminated: execution finished
  - Purpose of states
    - States help the OS schedule processes efficiently by representing lifecycle stages
- **Process Data Structures**
  - Process Control Block (PCB)
    - The PCB stores information the OS needs to manage each process
      - Contains process ID (PID), process state, CPU registers, scheduling information, memory management info, and I/O status
  - Role of PCB
    - Enables the OS to manage and switch between processes by saving and restoring state
- **Process API**
  - Typical process operations
    - Create process
    - Terminate process
    - Wait for process completion
    - Execute a new program
    - Get process information
  - Unix/Linux examples
    - fork, exec, wait, exit
- **Sharing Processor Among Processes**
  - CPU sharing overview
    - Since a CPU can run only one process at a time, the OS shares the CPU among processes
  - Mechanisms
    - CPU scheduling and time slicing
      - The OS rapidly switches between processes so they appear to run simultaneously
- **Linux Scheduling Implementation**
  - Scheduling classes
    - Real-time scheduling
    - Fair scheduling implemented by CFS
  - Run queues and load balancing
    - Each CPU has a run queue of runnable processes
    - Load balancing distributes processes across CPUs
  - Scalability
    - Design allows Linux to efficiently manage thousands of processes
- **Linux Completely Fair Scheduler (CFS)**
  - Goal of CFS
    - Ensure each process gets a fair share of CPU time
  - Key idea
    - CFS tracks virtual runtime (vruntime) for each process
      - The process with the smallest vruntime gets the CPU next
- **Multilevel Feedback Queue (MLFQ)**
  - MLFQ structure
    - Uses multiple queues with different priorities
  - Basic rules and behavior
    - Higher priority queues run first
    - Processes that use too much CPU move to lower priority queues
    - Interactive processes remain at higher priority
    - Periodic priority boosting prevents starvation
  - Adaptivity
    - MLFQ adapts scheduling based on process behavior
- **Basic Scheduling Algorithms**
  - First Come First Serve (FCFS)
    - Processes execute in arrival order
  - Shortest Job First (SJF)
    - Process with the shortest execution time runs first
  - Round Robin (RR)
    - Each process receives a fixed time slice
  - Priority Scheduling
    - Processes run based on assigned priority levels
- **Process Scheduling**
  - Scheduler role
    - Determines which process gets CPU time by selecting from the ready queue
  - Goals of scheduling
    - Maximize CPU utilization
    - Minimize waiting time
    - Improve responsiveness
    - Ensure fairness
- **Linux Implementation of Threads**
  - Pthreads model
    - Linux uses POSIX Threads (Pthreads)
      - Threads are treated as lightweight processes that share the same address space
  - Implementation detail
    - Each thread has its own task structure
    - Linux uses the clone system call internally to create threads
- **Multithreading Models**
  - Many-to-One
    - Many user threads mapped to one kernel thread
      - Efficient but prevents parallelism on multiple cores
  - One-to-One
    - Each user thread mapped to one kernel thread
      - Supports parallel execution across cores
  - Many-to-Many
    - Many user threads mapped to many kernel threads
      - Combines benefits of both models
- **Multithreading Benefits**
  - Advantages overview
    - Responsiveness: applications remain responsive during heavy tasks
    - Resource sharing: threads share memory and resources within the same process
    - Economy: creating threads is cheaper than creating processes
    - Scalability: threads can utilize multiple CPU cores
- **Threads and Concurrency**
  - Thread definition
    - A thread is the smallest unit of execution within a process
  - Shared and private resources
    - Threads within the same process share code, data, and files
      - Each thread has its own program counter, stack, and registers
  - Concurrency benefit
    - Multiple threads allow concurrent execution within processes
- **Linux Kernel Process Management**
  - Key features
    - Each process has a task structure
    - Processes are identified using PID
    - Supports process hierarchy with parent-child relationships
  - Management mechanisms
    - Linux maintains process lists and scheduling queues to manage execution efficiently
- **System Boot Sequence**
  - Typical boot steps
    - Power on
    - BIOS/UEFI initializes hardware
    - Bootloader is loaded
    - Bootloader loads the OS kernel
    - Kernel initializes system components
    - User environment starts
  - Example bootloader
    - GRUB for Linux
- **Context Switching**
  - What context switching is
    - Occurs when the CPU switches from executing one process to another
  - Steps in a context switch
    - Save the current process state in its PCB
    - Load the next process state
    - Resume execution
  - Purpose
    - Allows multiple processes to share the CPU by preserving and restoring states
- **User Mode and Kernel Mode**
  - Execution modes
    - User mode for applications with limited hardware access
    - Kernel mode for the OS with full hardware access
  - Interaction between modes
    - Programs request kernel services using system calls
      - Kernel mode executes critical system operations while user mode cannot perform privileged operations

SUBMITTED BY

Group 1

ATP24CS001 – AADINATH A AJAY RAJAGOPALAN
ATP24CS005 – ABHILASH S
ATP24CS009 – ADHEEBA SHERIN A
ATP24CS013 – ADITHI NARAYAN V
ATP24CS017 – AJNA H
ATP24CS021 – ALSHIFA K
ATP24CS025 – ANEESH R
ATP24CS029 – ANJANA V
ATP24CS033 – ARATHY S
ATP24CS037 – ASWATHY S NAIR
ATP24CS041 – BHAVYA DAS S
ATP24CS045 – DEEPAK K P
ATP24CS049 – DRISWAL K DEEPESH
ATP24CS053 – GOKUL KRISHNAN M
ATP24CS057 – JASMINE K
ATP24CS061 – K S ARUNGHOSH